OSSP (Operating Systems And Systems Programming) - 25CS2104E

Roll No. : 2520030027

Name: Venkata Sai Krishna

SEC-8

LAB InSEM EXAM:

Experiment-01

Execution:

```
venkatsaikrishna5@VSK-HP:~$ mkdir program1
mkdir: program1: File exists
venkatsaikrishna5@VSK-HP:~$ cd program1
venkatsaikrishna5@VSK-HP:~/program1$ nano pipe.c
venkatsaikrishna5@VSK-HP:~/program1$ ./pipe
Parent sent: Hello from parent!
Child received: Hello from parent!
```

Code:

```
 GNU nano 8.7.1

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <stdlib.h>

int main() {
    int fd[2];
    pid_t pid;
    char message[] = "Hello from parent!";
    char buffer[100];

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid > 0) {
        close(fd[0]);

        write(fd[1], message, strlen(message) + 1);
        printf("Parent sent: %s\n", message);

        close(fd[1]);
        wait(NULL);
    } else {
        close(fd[1]);

        read(fd[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);

        close(fd[0]);
        exit(0);
    }

    return 0;
}
```

Experiment-02:

Execution:

```
venkatsaikrishna5@VSK-HP:~$ cd program2
venkatsaikrishna5@VSK-HP:~/program2$ gcc signal.c -o signal
venkatsaikrishna5@VSK-HP:~/program2$ ./signal
Child process started. PID = 593
Child is waiting for a signal...
Parent sending SIGTERM to child...
Child process terminated by SIGTERM.
```

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child process started. PID = %d\n", getpid());
        printf("Child is waiting for a signal...\n");

        while (1) {
            pause();
        }
    } else {
        sleep(1);

        printf("Parent sending SIGTERM to child...\n");

        if (kill(pid, SIGTERM) == -1) {
            perror("kill");
            return 1;
        }

        wait(NULL);
        printf("Child process terminated by SIGTERM.\n");

        exit(0);
    }

    return 0;
}
```